Assignment 6

Question 1. (a)

Implement Gaussian Naïve Bayes Classifier on the Iris dataset from sklearn.datasets using
(a) Step-by-step implementation.

Solution

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split

iris_df = datasets.load_iris()
X = iris_df.data
y = iris_df.target

def mean(X):
    return np.mean(X, axis=0)

def variance(X):
    return np.var(X, axis=0)

def fit(X, y):
    model = {}
    model["classes"] = np.unique(y)
    model["mean"] = {}
    model["var"] = {}
    model["prior"] = {}

    for c in model["classes"]:
        X_c = X[y == c]
        model["mean"][c] = mean(X_c)
        model["var"][c] = variance(X_c)
        model["prior"][c] = X_c.shape[0] / X.shape[0]

    return model

def gaussian_prob(x, mean, var):
    exponent = np.exp(-((x - mean) ** 2) / (2 * var))
    return (1 / np.sqrt(2 * np.pi * var)) * exponent

def predict(model, X):
    y_pred = []
    for x in X:
        class_probs = {}
        for c in model["classes"]:
            prior = np.log(model["prior"][c])
            likelihood = np.sum(
                np.log(gaussian_prob(x, model["mean"][c], model["var"][c]))
            )
            class_probs[c] = prior + likelihood
        y_pred.append(max(class_probs, key=class_probs.get))
    return np.array(y_pred)

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)
model = fit(X_train, y_train)
y_pred = predict(model, X_test)

print("Predicted labels:", y_pred)
accuracy = np.mean(y_pred == y_test)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

OUTPUT

```
Predicted labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 100.00%
```

QUESTION 1. (B)

Implement Gaussian Naïve Bayes Classifier on the Iris dataset from sklearn.datasets using
(b) In-built function

SOLUTION

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB

data = datasets.load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = GaussianNB()
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)

y_pred = model.predict(X_test)
print("Predicted labels:", y_pred)
print(f"Accuracy: {accuracy * 100}%")
```

OUTPUT

```
Predicted labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 100.0%
```

Question 2

Explore about GridSearchCV toot in scikit-learn. This is a tool that is often used for tuning hyperparameters of machine learning models. Use this tool to find the best value of K for K-NN Classifier using any dataset.

Solution

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
param_grid = {"n_neighbors": np.arange(1, 10)}
print(param_grid)
knn = KNeighborsClassifier()

grid_search = GridSearchCV(knn, param_grid, cv=5)
grid_search.fit(X_train, y_train)
best_k = grid_search.best_params_["n_neighbors"]
print(f"Best value of K: {best_k}")

best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train, y_train)
accuracy = best_knn.score(X_test, y_test)
print(f"Accuracy with best K: {accuracy * 100}%")
```

Output

```
{'n_neighbors': array([1, 2, 3, 4, 5, 6, 7, 8, 9])}
Best value of K: 3
Accuracy with best K: 100.0%
```